

Identity and Access Management (IAM) for AI Systems

Controlling who can access what in your AI infrastructure



Why IAM Matters in AI Infrastructure



Protects Sensitive Data

Secures training corpora, vector databases, and feature stores from unauthorized access



Restricts Model Access

Controls who can train, serve, and fine-tune your models



Prevents Resource Misuse

Limits who can provision and use expensive GPU/compute resources



Ensures Compliance

Fundamental to meeting GDPR, HIPAA, SOC2 and other regulatory requirements

IAM Core Concepts



Identity

Users, service accounts, and roles that interact with your AI systems

Authentication

Verifying identity through SSO, API keys, certificates, and tokens

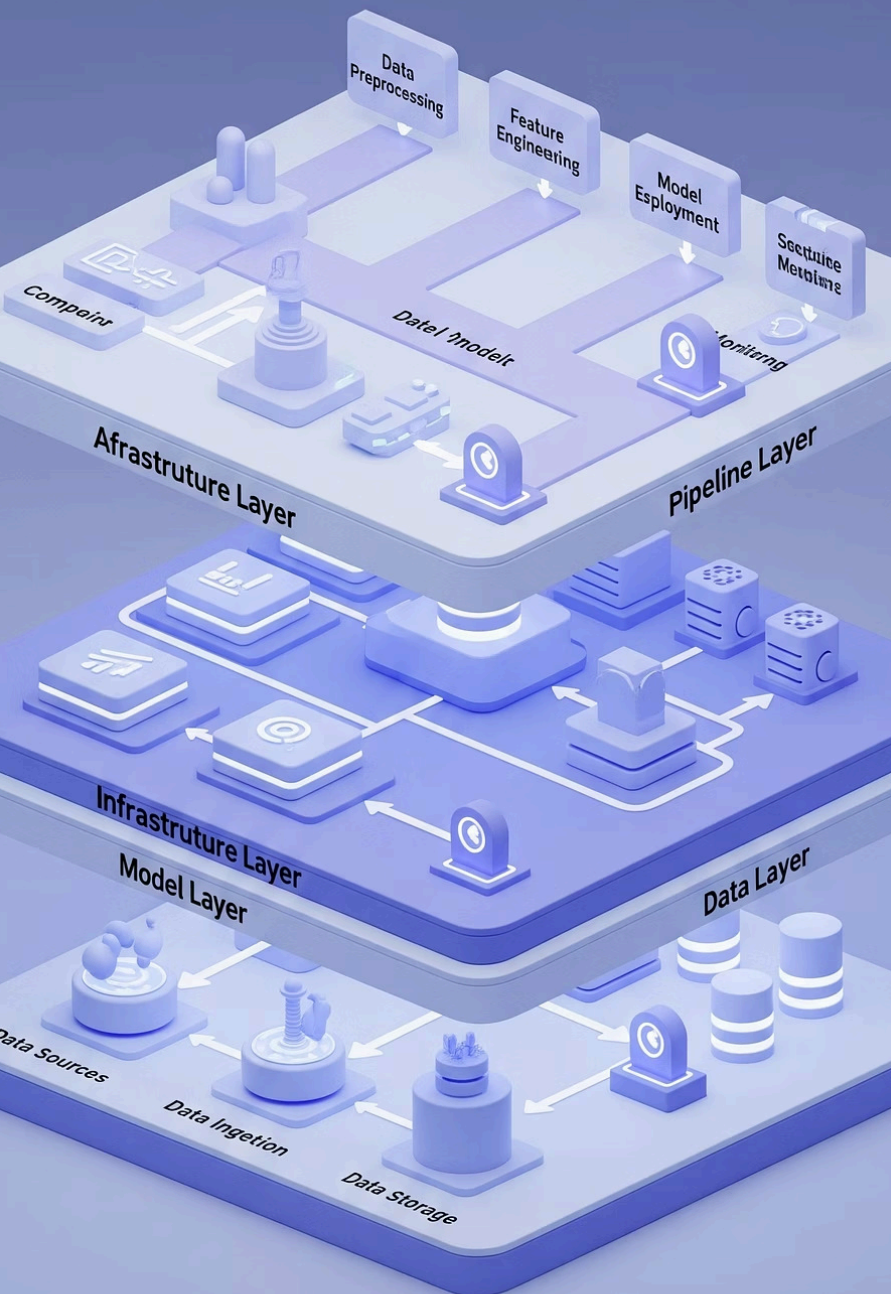
Authorization

Enforcing what an authenticated identity can access or modify

Principle of Least Privilege

Granting only the minimum permissions necessary for the task

IAM in AI Workflows



Data Layer

Controls who can read/write to:

- Feature stores
- Object storage (training data)
- Vector databases
- Data warehouses

Model Layer

Determines who can:

- Deploy models to production
- Query models via APIs
- Fine-tune existing models
- Access model weights

Infrastructure Layer

Limits who can:

- Provision GPU instances
- Scale compute clusters
- Configure networking
- Access compute logs

Pipeline Layer

Restricts who can:

- Push code to CI/CD
- Modify ML pipelines
- Trigger training jobs
- Deploy to production

Cloud IAM Examples

AWS IAM

- IAM roles and policies
- S3 bucket access controls
- SageMaker execution roles
- Resource-based policies

GCP IAM

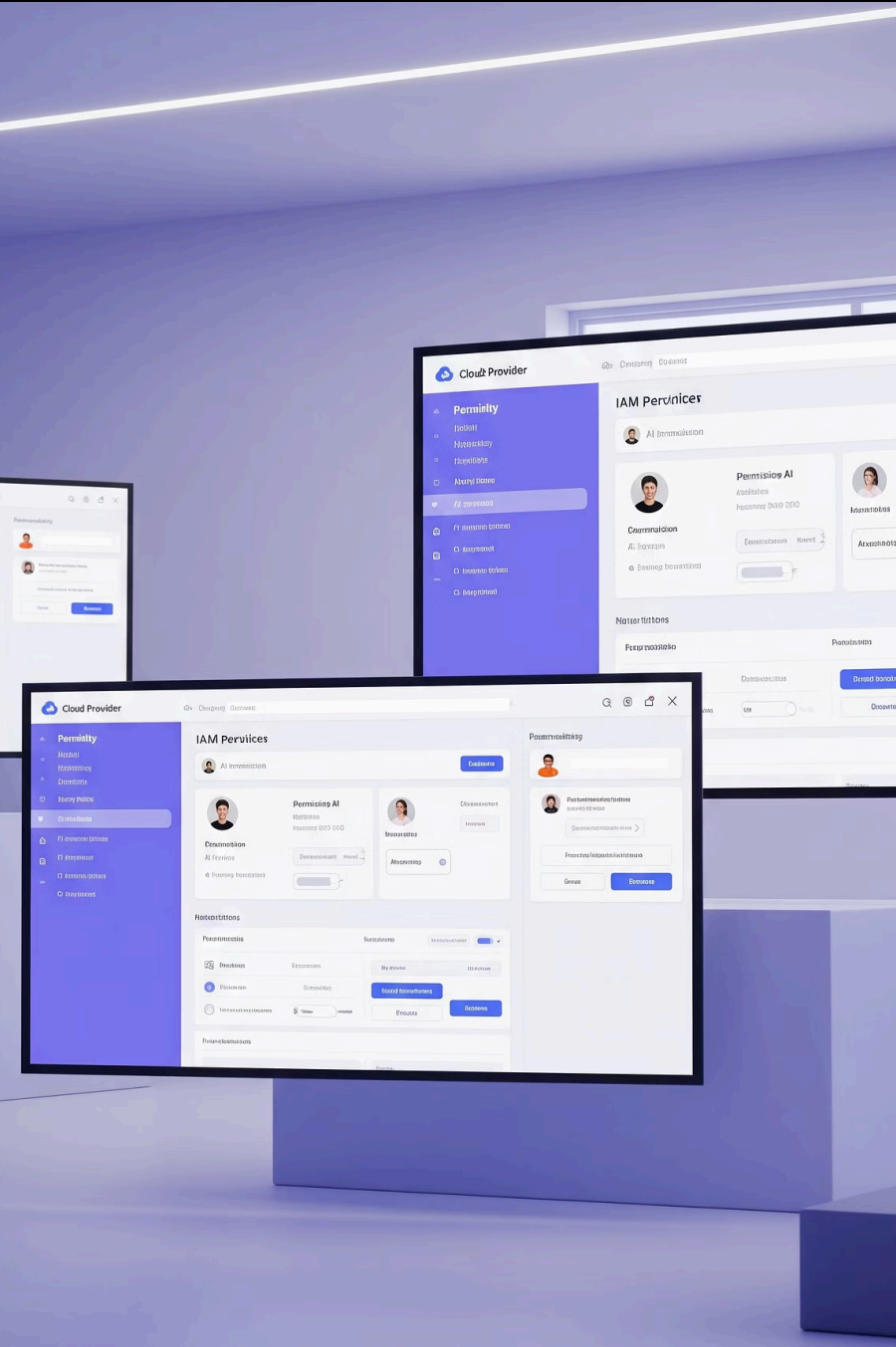
- Service accounts
- Vertex AI permissions
- Custom IAM roles
- VPC Service Controls

Azure RBAC

- Role-Based Access Control
- Storage access keys
- ML workspace roles
- Managed identities

⚠ Best Practice

Map users to scoped policies with specific permissions. Avoid wildcards (*) that grant excessive access.



Service Accounts & Workload Identity

Use Service Accounts

Replace human users with dedicated service identities for automated processes

Implement Workload Identity Federation

Allow Kubernetes pods, cloud functions, or containers to inherit cloud roles dynamically

Automate Credential Rotation

Set up automatic rotation of any service account keys that can't be eliminated

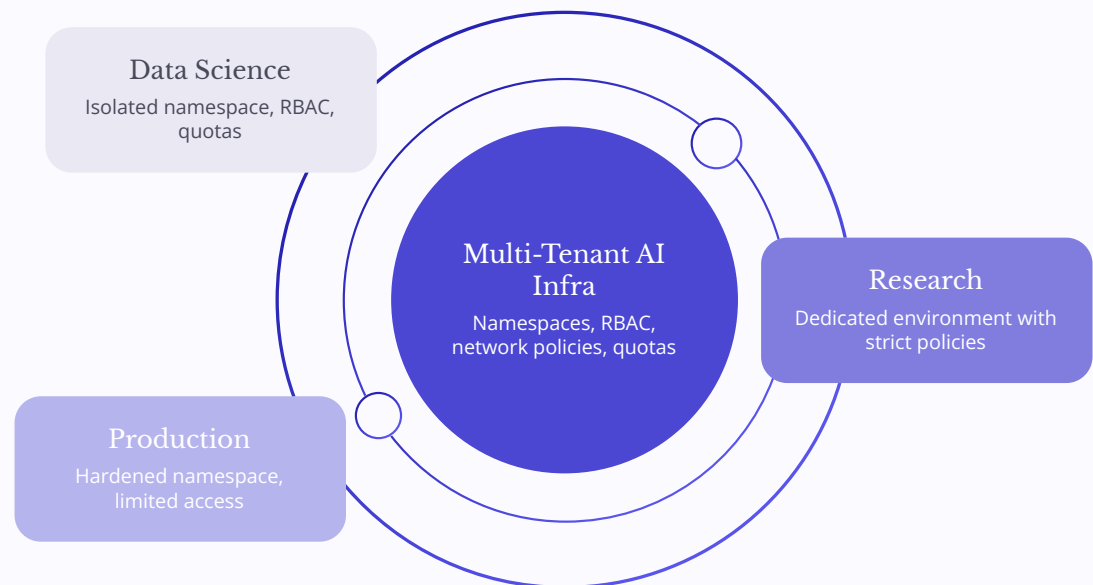
Never Embed Credentials

Avoid hardcoding credentials in code, containers, or CI/CD pipelines



Example: A training pod in Kubernetes automatically receives temporary AWS credentials through IAM Roles for Service Accounts (IRSA)

Multi-Tenant & Team Isolation



Isolation Strategies

- Kubernetes namespaces for team separation
- RBAC policies for granular access control
- Network policies to prevent cross-talk
- Resource quotas to ensure fair compute allocation

Business Benefits

- Prevents accidental data leakage between teams
- Stops GPU resource hogging by one team
- Enables charge-back and resource tracking
- Simplifies auditing and compliance

Secrets Management Integration



IAM Authentication

Verified identities request access to secrets



Secret Store

Vault, AWS KMS, GCP Secret Manager, Azure Key Vault



Protected Assets

API keys, certificates, encryption keys, database credentials

IAM ties directly into secret management systems, enforcing **encryption at rest and in transit**, centralizing **key rotation and audit logs**, and preventing secrets from being hardcoded in repositories or pipelines.

Monitoring & Auditing IAM

1

Access Logging

Track who accessed what resources and when

- CloudTrail, StackDriver, Azure Monitor
- SIEM integration for centralized logs

2

Anomaly Detection

Alert on suspicious patterns

- Off-hours resource provisioning
- Unusual model access patterns
- Excessive data downloads

3

Regular Audits

Perform systematic reviews

- Remove stale users and roles
- Revoke unused permissions
- Enforce MFA for privileged accounts



Comprehensive monitoring creates a clear audit trail for compliance requirements while enabling rapid response to security incidents.

IAM Threat Scenarios

Accidental Data Exposure

Developer mistakenly configures an S3 bucket with public read access, exposing proprietary training data to the internet.

Mitigation: Default-deny policies, S3 Block Public Access, IAM policy validation

Privilege Escalation

Malicious insider exploits loose permissions to gain GPU admin role, then uses compute resources for unauthorized cryptocurrency mining.

Mitigation: Strict RBAC, permission boundaries, just-in-time access

Compromised Service Account

Attacker obtains service account key from leaked code repository, uses it to exfiltrate valuable training data.

Mitigation: Workload identity, short-lived tokens, secret scanning

Pipeline Tampering

Over-privileged ML pipeline is compromised, allowing attacker to modify production models with malicious behavior.

Mitigation: Segregation of duties, approval workflows, model validation



IAM Best Practices

Policy Design

- Enforce least privilege + separation of duties
- Use groups and roles over individual permissions
- Break down monolithic policies into specific scopes
- Document policy decisions and exceptions

Technical Implementation

- Rotate & expire credentials automatically
- Use short-lived tokens instead of static keys
- Integrate IAM into CI/CD & IaC templates
- Conduct regular compliance reviews + audits

i The most effective IAM strategies balance security with developer productivity. Avoid creating policies so restrictive that teams resort to workarounds.

Key Takeaways

1

First Line of Defense

IAM forms the foundation of your AI security strategy, controlling access to all resources

3

Critical Assets Protected

Well-implemented IAM safeguards your data, models, and compute resources from misuse

4

Modern Approaches

Leverage service accounts, RBAC, and workload identity for secure, frictionless access

∞

Continuous Process

Regular audits and monitoring keep your IAM strategy effective and compliant as systems evolve

Questions?